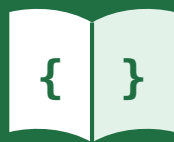

Module 01 — Spring Core Container: IoC and Dependency Injection

Lesson Notes

Spring Boot Edu



2026-09-23

Contents

1. Learning Goals	2
2. Concepts	2
2.1 Inversion of Control (IoC)	2
2.2 Dependency Injection (DI)	2
2.3 How Are Beans Found?	3
2.4 Container Startup, Step by Step	3
3. Step-by-Step Examples	3
3.1 Constructor and Setter Injection	3
3.2 @Component, @Bean and the @Configuration Proxy Mode	5
3.3 Bean Scopes	6
3.4 Lifecycle Callbacks	8
3.5 Profiles and Conditional Beans	9
3.6 Programmatic Registration with BeanRegistrar (Spring Framework 7)	10
3.7 Application Events	12
3.8 Cross-Cutting Concerns with AOP	13
4. Common Mistakes and Best Practices	14
5. Summary	15
6. Further Reading	15

1. Learning Goals

By the end of this module you will be able to:

- Explain Inversion of Control (IoC) and Dependency Injection (DI)
- Choose between constructor and setter injection, and avoid field injection
- Tell `@Component` from `@Bean` and understand the proxy behaviour of `@Configuration` classes
- Know singleton and prototype scopes, the “prototype inside a singleton” trap and its fix
- Use bean lifecycle callbacks (`@PostConstruct`, `@PreDestroy`, `SmartLifecycle`)
- Select beans per environment with profiles and `@ConditionalOnProperty`
- Register beans programmatically with `BeanRegistrar`, new in Spring Framework 7
- Publish and listen to application events
- Separate cross-cutting concerns with an aspect

Prerequisites: Module 00 (setup, modern Java) · **Estimated time:** 4 hours

2. Concepts

2.1 Inversion of Control (IoC)

In classic code an object creates what it needs (`new BookCatalog()`). With IoC this control is inverted: the **Spring container** creates the objects, wires them together and manages their lifecycle. Your class only states what it needs.

Every object managed by the container is called a **bean**. In a Spring Boot application the container is the `ApplicationContext` created by `SpringApplication.run(...)`.

2.2 Dependency Injection (DI)

DI is how IoC is put into practice: a bean receives its dependencies from the outside. There are three ways:

Style	When?
Constructor injection	The default choice. Required dependencies, <code>final</code> fields, testing without Spring
Setter injection	Optional dependencies
Field injection (<code>@Autowired</code> on a field)	Don't. Hard to test, hidden dependencies, fields cannot be <code>final</code>

2.3 How Are Beans Found?

- **Component scanning:** the package of the `@SpringBootApplication` class and its sub-packages are scanned. Classes annotated with `@Component`, `@Service`, `@Repository` or `@Controller` become beans.
- **@Bean methods:** the return values of methods in a `@Configuration` class become beans. Used for classes whose source you cannot change (e.g. `java.time.Clock`).
- **Programmatic registration:** with a `BeanRegistrar`, in plain Java code with loops and conditions (section 3.6).

2.4 Container Startup, Step by Step

1. Bean definitions are collected (scanning, `@Bean`, registrars); conditions (`@Profile`, `@Conditional...`) are evaluated.
2. Singleton beans are created and their dependencies injected.
3. `@PostConstruct` methods run; proxies (AOP) are created where needed.
4. `SmartLifecycle` beans start, `ApplicationRunners` run, the application is ready.
5. On shutdown `SmartLifecycle` beans stop and `@PreDestroy` methods run.

i Note

Singleton is Spring's default scope: the container creates **one** instance of each bean and everybody shares it. That is why singleton beans must not hold mutable state, or must be thread-safe.

3. Step-by-Step Examples

A tour (LessonTour) runs every example of the module in lesson order:

```
./mvnw -pl modules/01-core-container/lesson spring-boot:run
```

Each section prints its output under a `== 3.x` heading. All examples live below the `com.springbootedu.corecontainer` package.

3.1 Constructor and Setter Injection

Goal: take required dependencies through the constructor and optional ones through a setter.

`BookService` receives two required dependencies through its constructor. With a single constructor no `@Autowired` is needed:

```
@Service
public class BookService {

    private final BookCatalog catalog;           // final: set once, never null
    private final ApplicationEventPublisher events;

    // A single constructor needs no @Autowired: Spring calls it and passes the beans.
    public BookService(BookCatalog catalog, ApplicationEventPublisher events) {
        this.catalog = catalog;
        this.events = events;
    }
}
```

ReportPrinter uses a ReportFooter bean **if one exists**. Thanks to `@Autowired(required = false)` the setter is simply not called when there is no such bean:

```
@Component
public class ReportPrinter {

    private final BookCatalog catalog;           // required → constructor
    private @Nullable ReportFooter footer;       // optional → setter

    public ReportPrinter(BookCatalog catalog) {
        this.catalog = catalog;
    }

    @Autowired(required = false)                // skipped when no ReportFooter
    ⇨ bean exists
    public void setFooter(ReportFooter footer) {
        this.footer = footer;
    }
}
```

Expected output:

```
== 3.1 Constructor & setter injection
... BookService.findByAuthor took 139 µs
Joshua Bloch: [Effective Java, Java Puzzlers]
Kitap sayısı / Book count: 4
- Effective Java
- Java Puzzlers
- Modern Java in Action
- Spring in Action
```

Its test: `book/BookServiceTest` tests the service without Spring at all, just new `BookService(...)`. That is the biggest win of constructor injection. `book/ReportPrinterTest` covers both cases, with and without a footer bean.

3.2 @Component, @Bean and the @Configuration Proxy Mode

Goal: register our own classes with @Component and other people's classes with @Bean, and tell the "full" and "lite" modes of @Configuration classes apart.

java.time.Clock belongs to the JDK, so we cannot put @Component on it. We register it with a @Bean method instead:

```
@Configuration(proxyBeanMethods = false)
public class AppConfig {

    @Bean
    Clock clock() {
        return Clock.systemDefaultZone(); // java.time.Clock: a JDK class, no
        ↪ @Component possible
    }
}
```

In the default (**full**) mode Spring creates a CGLIB subclass (a proxy) of the @Configuration class. When one @Bean method calls another, the proxy intercepts the call and returns the **existing singleton**:

```
@Configuration // proxyBeanMethods = true (default)
static class FullModeConfiguration {

    @Bean
    TaxRate fullModeTaxRate() {
        return new TaxRate(new BigDecimal("0.20"));
    }

    @Bean
    PriceCalculator fullModePriceCalculator() {
        // The CGLIB proxy intercepts this call and returns the existing singleton bean.
        return new PriceCalculator(fullModeTaxRate());
    }
}
```

With proxyBeanMethods = false (**lite** mode) no proxy is created. Startup is faster and it works well with native images, but a method call is now just a plain Java call:

```
@Configuration(proxyBeanMethods = false) // no proxy: faster startup, plain
    ↪ Java calls
static class LiteModeConfiguration {

    @Bean
    TaxRate liteModeTaxRate() {
```

```
        return new TaxRate(new BigDecimal("0.20"));
    }

    @Bean
    PriceCalculator liteModePriceCalculator() {
        // PITFALL: without a proxy this is a plain method call → a second, unmanaged
        // ↪ TaxRate.
        return new PriceCalculator(liteModeTaxRate());
    }

    @Bean
    PriceCalculator liteModeSafePriceCalculator(TaxRate liteModeTaxRate) {
        // Correct lite-mode style: receive the bean as a method parameter.
        return new PriceCalculator(liteModeTaxRate);
    }
}
```

Expected output:

```
== 3.2 @Configuration full vs lite mode
full mode shares the TaxRate bean: true
lite mode shares the TaxRate bean: false
```

Its test: configuration/ProxyModeTest verifies all three cases (full, the lite pitfall, correct lite usage) with an ApplicationContextRunner.

★ Tip

Spring Boot's own auto-configuration classes use lite mode. If you wire @Bean methods together **through method parameters**, lite mode is safe for you too.

3.3 Bean Scopes

Goal: see the difference between singleton and prototype, and the “prototype inside a singleton” trap.

ShoppingCart holds state (the items in the cart). Every customer needs their own cart, so it uses the prototype scope: every lookup from the container creates a new object:

```
@Component
@Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
public class ShoppingCart {

    private final List<String> isbnns = new ArrayList<>(); // state → must not be shared
}
```

The trap: when a prototype bean is injected into a singleton, the singleton is created once, so the prototype is created **only once** as well. Every customer then shares the same cart:

```
@Service
public class BrokenCheckoutService {

    private final ShoppingCart cart;          // injected once, when this singleton is
    ↪ created

    public BrokenCheckoutService(ShoppingCart cart) {
        this.cart = cart;
    }

    public ShoppingCart startCheckout() {
        return cart;                        // every customer gets the SAME cart!
    }
}
```

The fix: inject an `ObjectProvider` that creates the object, instead of the object itself, and ask it for a new instance whenever you need one:

```
@Service
public class CheckoutService {

    private final ObjectProvider<ShoppingCart> carts;

    public CheckoutService(ObjectProvider<ShoppingCart> carts) {
        this.carts = carts;
    }

    public ShoppingCart startCheckout() {
        return carts.getObject();           // a new cart on every call
    }
}
```

Expected output:

```
== 3.3 Scopes
prototype in singleton → same cart twice: true
ObjectProvider → same cart twice: false
```

Its test: `scope/CheckoutServiceTest`

i Note

Web applications also have request and session scopes. We will meet them in Module 03 (Web MVC).

3.4 Lifecycle Callbacks

Goal: put code that must run when a bean is ready, and cleanup code for shutdown, in the right place.

Dependencies are available in the constructor, but putting the “preparation” work (e.g. building an index) into a `@PostConstruct` method makes the intent clear. `@PreDestroy` runs when the context closes:

```
@Component
public class TitleIndex {

    private final BookCatalog catalog;
    private final Map<String, String> titlesByLowerCase = new ConcurrentHashMap<>();

    public TitleIndex(BookCatalog catalog) {
        this.catalog = catalog;           // 1. constructor: dependencies are injected
    }

    @PostConstruct
    void build() {                        // 2. after injection: safe to use dependencies
        catalog.findAll().stream()
            .map(Book::title)
            .forEach(title -> titlesByLowerCase.put(title.toLowerCase(Locale.ROOT),
                ↪ title));
    }

    @PreDestroy
    void clear() {                       // 3. on context close: release resources
        titlesByLowerCase.clear();
    }
}
```

A background job (e.g. periodic synchronisation) should start only after **all** beans are ready and stop cleanly on shutdown. That is what `SmartLifecycle` is for:

```
@Component
public class InventorySync implements SmartLifecycle {

    private static final Logger log = LoggerFactory.getLogger(InventorySync.class);

    private final Clock clock;
```

```
private volatile boolean running;
private volatile @Nullable Instant startedAt;

public InventorySync(Clock clock) {
    this.clock = clock;
}

@Override
public void start() { // called once the context has refreshed
    startedAt = clock.instant();
    running = true;
    log.info("Stok senkronizasyonu başladı / Inventory sync started");
}

@Override
public void stop() { // called when the context closes
    running = false;
    log.info("Stok senkronizasyonu durdu / Inventory sync stopped");
}

@Override
public boolean isRunning() {
    return running;
}
```

Expected output:

```
... InventorySync : Stok senkronizasyonu başladı / Inventory sync started
== 3.4 Lifecycle
TitleIndex.search("java") = [Effective Java, Java Puzzlers, Modern Java in Action]
InventorySync running: true
... InventorySync : Stok senkronizasyonu durdu / Inventory sync stopped
```

Its test: lifecycle/LifecycleTest checks that the index is cleared and the sync stops once the context is closed.

3.5 Profiles and Conditional Beans

Goal: use different beans with the same code, depending on the environment or configuration.

@ConditionalOnProperty creates a bean depending on the value of a property. matchIfMissing = true makes this option the default:

```

@Service
@ConditionalOnProperty(name = "bookstore.recommendations.strategy", havingValue =
    ↪ "popular", matchIfMissing = true)
public class PopularBooksRecommendation implements RecommendationService {

```

A bean annotated with `@Profile` exists only while that profile is active. It is ideal for loading sample data in development only:

```

@Component
@Profile("dev") // run with: --spring.profiles.active=dev
public class DevSampleData implements ApplicationRunner {

    private final BookCatalog catalog;

    public DevSampleData(BookCatalog catalog) {
        this.catalog = catalog;
    }

    @Override
    public void run(@Nullable ApplicationArguments args) {
        catalog.save(new Book("978-605-000-001-1", "Kürk Mantolu Madonna", "Sabahattin
    ↪ Ali", new BigDecimal("45.00")));
        catalog.save(new Book("978-605-000-002-8", "Tutunamayanlar", "Oğuz Atay", new
    ↪ BigDecimal("110.00")));
    }
}

```

Run it: with the default settings, then with the dev profile and the budget strategy:

```

./mvnw -pl modules/01-core-container/lesson spring-boot:run \
-Dspring-boot.run.arguments="--spring.profiles.active=dev
    ↪ --bookstore.recommendations.strategy=budget"

```

Expected output (second run):

```

== 3.5 Profiles & conditions
active profiles: [dev]
BudgetBooksRecommendation → [Kürk Mantolu Madonna, Java Puzzlers, Effective Java]

```

Its tests: `condition/RecommendationConditionTest`, `condition/DevSampleDataTest`

3.6 Programmatic Registration with BeanRegistrar (Spring Framework 7)

Goal: decide which beans to create with plain Java code (loops, if).

The `BeanRegistrar` interface arrived with Spring Framework 7. This one reads the list of channels from the configuration and registers one bean per channel. It is much simpler than the old `BeanDefinitionRegistryPostProcessor` approach and works with AOT/native images:

```
public class NotificationChannelsRegistrar implements BeanRegistrar {

    private static final Map<String, Supplier<NotificationChannel>> KNOWN_CHANNELS =
        ↪ Map.of(
            "email", EmailChannel::new,
            "sms", SmsChannel::new,
            "push", PushChannel::new);

    @Override
    public void register(BeanRegistry registry, Environment env) {
        String[] configured = env.getProperty("bookstore.notifications.channels",
            ↪ String[].class, new String[] {"email"});
        List<String> names = Ar-
            ↪ rays.stream(configured).map(String::trim).filter(KNOWN_CHANNELS::containsKey).toList();

        for (String name : names) {
            Supplier<NotificationChannel> factory = KNOWN_CHANNELS.get(name);
            registry.registerBean(name + "Channel", NotificationChannel.class,
                spec -> spec.supplier(context -> factory.get()));
        }
    }
}
```

A registrar is added to a `@Configuration` class with `@Import`. Injecting `List<NotificationChannel>` collects **all** registered channels:

```
@Configuration(proxyBeanMethods = false)
@Import(NotificationChannelsRegistrar.class)
public class NotificationConfiguration {

    @Bean
    NotificationService notificationService(List<NotificationChannel> channels) {
        return new NotificationService(channels);
    }
}
```

Expected output (with channels: email,push in application.yaml):

```
== 3.6 BeanRegistrar
[[email] Kampanya başladı!, [push] Kampanya başladı!]
```

Its test: registrar/NotificationChannelsRegistrarTest

3.7 Application Events

Goal: announce the result of some work without knowing who listens (loose coupling).

After adding a book, BookService publishes a BookAddedEvent:

```
public void add(Book book) {
    catalog.save(book);
    events.publishEvent(new BookAddedEvent(book)); // listeners run after this line
}
```

A listener declares the event it handles through its parameter type. @Order sets the order of several listeners:

```
@Component
public class NewBookAnnouncer {

    private final NotificationLog log;

    public NewBookAnnouncer(NotificationLog log) {
        this.log = log;
    }

    @EventListener
    @Order(1) // lower value → called earlier
    public void announce(BookAddedEvent event) {
        log.add("Yeni kitap / New book: " + event.book().title());
    }
}
```

The condition attribute is a SpEL expression. This listener only runs for books priced 100 or more:

```
@Component
public class PremiumBookListener {

    private final NotificationLog log;

    public PremiumBookListener(NotificationLog log) {
        this.log = log;
    }

    @EventListener(condition = "#event.book().price() >= 100")
    @Order(2)
    public void onPremiumBook(BookAddedEvent event) {
        log.add("Premium kitap / Premium book: " + event.book().title());
    }
}
```

```
}  
}
```

Expected output:

```
== 3.7 Events  
Yeni kitap / New book: Spring Boot: Up and Running  
Premium kitap / Premium book: Spring Boot: Up and Running
```

Its test: event/BookEventListenersTest

! Important

By default `@EventListener` methods run **immediately, on the same thread**. To handle an event only after a database transaction has committed successfully, use `@TransactionalEventListener`. We will see it together with transactions in Module 05.

3.8 Cross-Cutting Concerns with AOP

Goal: keep code that concerns many classes, such as timing, out of the business logic.

An aspect measures the duration of methods annotated with `@LogExecutionTime`. `BookService.findByAuthor` carries that annotation:

```
@Aspect  
@Component  
public class ExecutionTimeAspect {  
  
    private static final Logger log = LoggerFactory.getLogger(ExecutionTimeAspect.class);  
  
    private final MethodTimings timings;  
  
    public ExecutionTimeAspect(MethodTimings timings) {  
        this.timings = timings;  
    }  
  
    @Around("@annotation(com.springbootedu.corecontainer.aop.LogExecutionTime)")  
    public Object measure(ProceedingJoinPoint joinPoint) throws Throwable {  
        String method = joinPoint.getSignature().getDeclaringType().getSimpleName() + "."  
            + joinPoint.getSignature().getName();  
        long start = System.nanoTime();  
        try {  
            return joinPoint.proceed();           // call the real method  
        } finally {
```

```
        long micros = (System.nanoTime() - start) / 1_000;
        timings.record(method);
        log.info("{} took {} μs", method, micros);
    }
}
```

Spring creates a **proxy** for `BookService`. Other beans receive this proxy instead of the real object, and every call passes through the aspect first.

Expected output:

```
... ExecutionTimeAspect : BookService.findByAuthor took 139 μs
== 3.8 AOP
measured methods: [BookService.findByAuthor]
```

Its test: `aop/ExecutionTimeAspectTest`

! Warning

In Spring Boot 4 the AOP starter is called `spring-boot-starter-aspectj`. The old `spring-boot-starter-aop` no longer exists.

4. Common Mistakes and Best Practices

! Warning

Self-invocation: if a class calls its own `@LogExecutionTime` method via `this.findByAuthor(...)`, the call does not go through the proxy and the aspect **does not run**. The same applies to `@Transactional`, `@Async` and `@Cacheable`.

- **Do:** take required dependencies through the constructor and make the fields `final`.
- **Don't:** use field injection. It hides dependencies, and the class cannot be tested without Spring.
- **Do:** keep stateful objects out of singletons. If you need a prototype, get it through an `ObjectProvider`.
- **Don't:** call one `@Bean` method from another in lite mode (`proxyBeanMethods = false`); ask for the bean as a parameter.
- **Do:** when an interface has several implementations, choose with `@ConditionalOnProperty` or `@Profile`, not with `if` blocks.
- **Don't:** build circular dependencies between constructors ($A \rightarrow B \rightarrow A$). Spring Boot rejects them by default, and they point to a design problem.
- **Do:** when you wonder why a bean was (not) created, run the application with `--debug` and read the conditions report.

5. Summary

- The Spring container creates objects (beans), wires them together and manages their lifecycle.
- Constructor injection is the default choice. Setter injection is only for optional dependencies.
- `@Component` is for your own classes, `@Bean` for other people's. In lite mode, take dependencies as method parameters.
- Singleton is the default. For stateful beans use prototype together with `ObjectProvider`.
- `@PostConstruct/@PreDestroy` and `SmartLifecycle` organise startup and shutdown work.
- `@Profile` and `@ConditionalOnProperty` select beans per environment. `BeanRegistrar` is for complex registration logic.
- Events decouple components from each other. AOP separates cross-cutting concerns from business code.

Next step: [Exercises](#)

6. Further Reading

- [Spring Framework — The IoC Container](#)
- [Spring Framework — Programmatic Bean Registration](#)
- [Spring Framework — Bean Scopes](#)
- [Spring Framework — Application Events](#)
- [Spring Framework — Aspect Oriented Programming](#)
- [Spring Boot — Profiles](#)